Heterogeneous Multiagent Task Allocation Based on Graph-Based Convolutional Assignment Neural Network

Ziyuan Ma, *Member, IEEE*, Huajun Gong, Jun Xiong, *Member, IEEE*, and Xinhua Wang

***Abstract*—Task allocation in complex multiagent systems involves assigning tasks to agents with varying capabilities to optimize overall performance. The challenge lies in selecting the most suitable agent for each task, considering the agents' heterogeneity and the intricate relationships between tasks. Traditional methods often fail to capture this complexity. To address these limitations, we propose the graph multiagent task allocation neural network (GMATANN), a novel approach utilizing a graph attention mechanism. GMATANN models the interactions between agents and tasks through a task-agent graph, where both agents and tasks are represented as nodes, and their associations are depicted as edges. The graph attention mechanism is crucial for capturing the key relationships and ensuring effective information flow between nodes. By learning attention weights, the network automatically identifies which agents are best suited for specific tasks. We employ a neural network framework based on this attention mechanism to train and evaluate the method. Simulation experiments demonstrate the effectiveness of GMATANN, achieving a task allocation accuracy of 92.3% and a reliability of 94.2%, outperforming traditional approaches. This innovative method offers a new strategy for complex task allocation in multiagent systems, providing an adaptive solution that selects suitable agents for diverse tasks, thereby enhancing system efficiency.**

***Index Terms*—Attention mechanism, graph neural network (GNN), multiagent, task allocation, autonomous aerial vehicles (AAVs), unmanned ground vehicles (UGVs), unmanned surface vehicles (USVs).**

I. Introduction

HETEROGENEOUS multiagent task allocation poses significant challenges in complex scenarios where agents with diverse capabilities must collaborate to accomplish various tasks [1], [2]. Real-world applications, such as urban disaster response and logistics management, highlight the importance of efficient and adaptive task allocation strategies [3], [4]. In such systems, agents like autonomous aerial vehicles (AAVs), unmanned ground vehicles (UGVs), and unmanned surface vehicles (USVs) operate under dynamic and resource-constrained conditions, requiring seamless coordination and effective utilization of their unique capabilities. Despite advancements in task allocation methodologies, existing approaches often struggle to address the complexities of these heterogeneous environments [5], [6], [7].

In multiagent systems, heterogeneity refers to the differences in the capabilities, operational environments, and roles of the involved agents. For example, in a system that includes AAVs and UGVs, heterogeneity is reflected in the distinct functionalities provided by each type of agent: AAVs are optimized for aerial tasks, such as surveillance, while UGVs are better suited for ground-based operations, such as transport and rescue [8], [9], [10]. These differences require task allocation methods to consider not only the scale of the system but also the diversity of the agents and tasks involved.

Centralized task allocation methods, while effective in small to moderately sized homogeneous systems, struggle to address the complexities introduced by heterogeneity. In a centralized approach, a single decision-making entity attempts to optimize task allocation across all agents based on a global perspective [11]. While this can theoretically provide a globally optimal solution, the process is computationally intensive, especially as the number of agents and the diversity of tasks increase. The challenge lies not just in the scale but in the nature of heterogeneity: a centralized system must account for different agent capabilities, varying task requirements, and the interactions between these elements. This makes the problem exponentially more complex, leading to slower decision making and potential inefficiencies as the system scales [12], [13], [14].

Traditional task allocation methods can be broadly categorized into centralized and distributed approaches. Centralized methods leverage a single decision-making entity with a global perspective of the system, enabling them to achieve global optimization. However, these approaches are often computationally intensive and suffer from scalability issues, particularly in large-scale systems with diverse agent capabilities. Additionally, the reliance on a centralized entity introduces single points of failure, making such methods less reliable in dynamic and high-risk environments. Distributed methods, on the other hand, delegate decision making to individual agents, allowing them to make autonomous decisions based on local information. While this decentralization improves scalability and robustness, it often results in suboptimal task allocations, as distributed systems lack the ability

Received 14 December 2024; revised 12 January 2025; accepted 24 January 2025. Date of publication 28 January 2025; date of current version 23 May 2025. *(Corresponding author: Huajun Gong.)*

Ziyuan Ma, Huajun Gong, and Xinhua Wang are with the Flight Control Laboratory, College of Automation Engineering, Nanjing University of Aeronautics and Astronautics, Nanjing 210016, China (e-mail: maziyuan@nuaa.edu.cn; ghj301@nuaa.edu.cn; xhwang@nuaa.edu.cn).

Jun Xiong is with the School of Internet of Things, Nanjing University of Posts and Telecommunications, Nanjing 210023, China (e-mail: xiongjun@njupt.edu.cn).

Digital Object Identifier 10.1109/JIOT.2025.3535641

to account for global dependencies and complex interrelationships between tasks and agents. These limitations underscore the need for a novel approach that balances global optimization with local adaptability in heterogeneous systems.

Distributed task allocation methods offer a different approach by decentralizing the decision-making process. Each agent makes decisions based on local information and interactions with nearby agents, which enhances scalability and robustness. However, when dealing with heterogeneous systems, these methods can fall short [15], [16], [17]. The local decision-making processes may not fully capture the global context, leading to suboptimal task allocation, particularly when tasks require coordinated efforts between different types of agents with varying capabilities. This limitation is amplified in heterogeneous systems, where the differences between agents are more pronounced, and effective task allocation requires a more integrated understanding of these differences [18], [19], [20].

Given the limitations of both centralized and distributed approaches in addressing heterogeneity, a new method that integrates the strengths of both paradigms is necessary. The proposed graph multiagent task allocation neural network (GMATANN) aims to bridge this gap by leveraging a graph-based representation that models the relationships between diverse agents and tasks. For centralized and distributed systems, we propose a complementary approach, characterized as a mixed hybrid paradigm. This approach combines the global perspective of centralized methods with the scalability and robustness of distributed methods. For example, consider an urban disaster response scenario involving AAVs, UGVs, and USVs. The GMATANN model constructs a graph where nodes represent the different agents and tasks, and edges represent the relationships and dependencies between them. By applying a graph attention mechanism, the model can dynamically adjust task allocation strategies based on the specific capabilities of each agent and the requirements of each task, effectively handling the heterogeneity in the system.

While the GMATANN model draws from both centralized and distributed methodologies, it does not strictly adhere to either paradigm. Instead, it offers a hybrid solution that can adapt to the specific needs of the system, whether that involves centralized decision making, distributed coordination, or a combination of both. This flexibility is particularly valuable in heterogeneous systems, where the nature of the tasks and the capabilities of the agents can vary widely. Through simulation experiments, we demonstrate the feasibility and effectiveness of this approach across different scenarios, showcasing its potential to address the challenges posed by heterogeneity in multiagent systems.

Recent advancements in multiagent systems have led to a proliferation of studies on task allocation strategies, reflecting the growing importance of optimizing collaboration in diverse, complex environments. These studies have primarily focused on centralized, distributed, and hybrid approaches, each offering unique benefits and challenges. However, the limitations of existing methods in addressing heterogeneity, scalability, and adaptability underscore the need for further research, forming the basis of the present study.

Centralized task allocation approaches have been widely studied for their potential to achieve global optimization. For instance, auction-based methods and mathematical programming models have been proposed to assign tasks to agents by considering global utility maximization. These methods are effective in environments with well-defined, static tasks and limited agent heterogeneity. However, they struggle with computational overhead and scalability as the number of agents and tasks increases, particularly in heterogeneous systems where task complexity varies significantly. Moreover, centralized approaches are vulnerable to single points of failure, making them less reliable in dynamic, high-risk scenarios, such as disaster response.

Distributed methods, on the other hand, allow agents to make autonomous decisions based on local information and peer-to-peer interactions. Reinforcement learning (RL)-based distributed models have been increasingly adopted, enabling agents to learn optimal task allocation strategies through interactions with their environment. For example, multiagent deep RL (MADRL) has been applied to address distributed task allocation problems, showing promising results in terms of scalability and robustness. However, distributed methods often fail to capture the global context, leading to suboptimal task allocation. Additionally, their reliance on local interactions can hinder effective coordination in systems with complex task interdependencies and varying agent capabilities.

To address the limitations of centralized and distributed methods, hybrid approaches have emerged, combining the strengths of both paradigms. Graph-based task allocation models, for instance, have shown potential in modeling the relationships between agents and tasks using graph neural networks (GNNs). These methods leverage the ability of GNNs to capture both local and global dependencies, making them particularly suitable for heterogeneous multiagent systems. Recent studies have explored the integration of GNNs with attention mechanisms, allowing task allocation models to focus on critical relationships within the system. While these approaches demonstrate improved adaptability and performance, they often lack the ability to handle dynamic task environments effectively, as most models rely on static graph representations or predefined task-agent mappings.

Another line of research has explored the use of evolutionary algorithms, such as genetic algorithms (GAs), to optimize task allocation in heterogeneous systems. These methods perform well in finding near-optimal solutions in large search spaces and are adaptable to dynamic changes. However, their computational complexity and convergence time limit their practical application in real-time scenarios.

Despite these advancements, several gaps remain in the literature. Existing methods often fail to simultaneously address the key challenges of heterogeneity, dynamic adaptation, and efficient resource utilization. Most studies focus on either centralized optimization or distributed learning without fully leveraging the complementary strengths of these approaches. Moreover, while graph-based methods provide a powerful framework for capturing relationships within a multiagent system, they are often not combined with dynamic learning techniques, such as RL, to enable real-time adaptability.

The limitations of traditional centralized and distributed methods, coupled with the emerging potential of graph-based hybrid approaches, highlight the need for innovative solutions. The present study builds on this foundation by proposing a GMATANN, which combines the strengths of GNNs and RL. This approach aims to address the identified gaps by providing a scalable, adaptable, and efficient framework for heterogeneous multiagent task allocation. By integrating global-local attention mechanisms and dynamic learning capabilities, GMATANN offers a novel solution to the challenges faced by current methods.

Unlike traditional centralized methods, which depend on a single decision-making entity and often face challenges with scalability and single points of failure, GMATANN distributes the decision-making process while retaining a global perspective. The graph attention mechanism enables GMATANN to capture and analyze the complex interdependencies between agents and tasks, allowing it to make globally informed decisions that are typically beyond the capabilities of purely distributed methods. This hybrid approach merges the robustness and scalability of distributed systems with the global optimization potential of centralized methods, making it particularly effective in managing heterogeneous multiagent systems. The main contributions of this article are as follows.

1) This article introduces a novel task allocation model, GMATANN, specifically designed to tackle the challenges of task allocation in heterogeneous multiagent systems. By incorporating a graph attention mechanism, GMATANN effectively captures the complex relationships between various agents and tasks, thereby enhancing the efficiency and accuracy of task allocation. Compared to traditional centralized task allocation methods, GMATANN demonstrates significant improvements in managing heterogeneity and complexity. Unlike distributed methods, GMATANN strikes a better balance between global optimality and the flexibility of local decision making.

2) This article proposes a method for constructing a task-agent graph, where nodes represent agents and tasks, and edges represent their associations. To address the issue of interesting centralized and distributed systems, we propose a complementary approach, characterized as a mixed hybrid paradigm. By applying a graph attention mechanism, GMATANN can automatically learn and assign attention weights to these relationships, thereby inferring the most suitable agents for specific tasks. Compared to existing methods, this mechanism significantly enhances the decision-making process in task allocation, especially in handling complex and dynamic scenarios.

3) This article demonstrates the ability to make adaptive response to different types of task assignments through GMATANN model by a series of simulation experiments. To handle task allocation in various heterogeneous scenarios, GMATANN model can integrate centralized and distributed systems, resulting in higher efficiency and faster focusing on the key task allocation. The results demonstrate that GMATANN outperforms traditional methods in both accuracy and efficiency of task allocation, improving 20% performance indicators, further proving its practical applicability and superiority in real-world multiagent systems.

Section II presents the assumed scenario for heterogeneous multiagent tasks. Section III focuses on the development of the heterogeneous multiagent model, covering key aspects, such as environmental modeling, intelligent agent modeling, and constraint modeling. Section IV introduces the design of the multiagent systems algorithm based on graph attention mechanism neural networks, referred to as the GMATANN algorithm. Finally, the model is tested and validated.

II. Algorithm Design for Multiagent Systems Based on GMATANN Model

While GNNs have made significant progress in addressing the heterogeneous multiagent assignment problem, capturing associations and interactions between intelligent agents and tasks, and optimizing task allocation strategies through embedded graph representations [21], [22], [23], there remain challenges that existing models do not fully address. In particular, most current GNN-based approaches struggle with effectively integrating temporal dynamics and context-specific variables, which are crucial for real-time task allocation in complex environments.

This article presents a novel algorithm that combines GNNs with an attention mechanism to enable joint collaborative task allocation in heterogeneous multiagent systems. Unlike previous works, our approach introduces three key innovations as follows.

Time Series Conditional Variable Self-Encoder: This component enhances the model's ability to capture temporal dependencies and context-specific conditions in task allocation, going beyond the static representations typically used in traditional GNN models. This allows for a more dynamic and responsive task allocation strategy.

Global-Local Fusion Feature Attention Mechanism: We introduce a novel attention mechanism that fuses global and local features, improving the model's ability to differentiate between tasks that require broader coordination and those that are locally focused. This contrasts with existing attention mechanisms, which often overlook the hierarchical nature of tasks and agent interactions.

RL Integration: By incorporating RL, our model not only learns optimal task allocation strategies from the past experiences but also adapts to changing environments in real-time. This integration differentiates our approach from previous methods that primarily rely on static or batch learning processes.

Through these innovations, our algorithm outperforms existing methods by better addressing the dynamic and heterogeneous nature of multiagent task allocation. By comparing our approach with related works (referenced literature), we demonstrate significant improvements in both efficiency and effectiveness, particularly in scenarios that require real-time decision making and adaptability.

In the task allocation and optimization of agent collaboration, GNN and RL can be combined. Fig. 1 shows the basic framework for optimizing task allocation using GNN and RL.

Using agents and tasks as nodes in the graph, construct edges based on their relationships. The attributes of agents

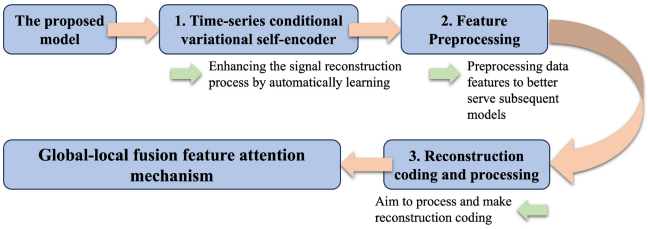


Fig. 1. Main processing flowchart of the system model.

and tasks can be represented by node features, such as the speed and mobility of agents, the type and requirements of tasks, etc. Edges can represent the connections and dependencies between agents and tasks, such as the degree of matching between agents and tasks, and the collaborative relationship between tasks. Using GNN for information transmission and node updates: through multilayer GNN networks for information transmission, nodes can obtain information and relationships from surrounding nodes. The GNN model can combine node features and edge information for node updates and representation learning. This can better represent and understand the positions and relationships of agents and tasks in the graph [24], [25], [26].

Task Allocation Optimization of RL: Task allocation is regarded as an RL problem, and agents learn the best task allocation strategy by interacting with the environment. Define the state representation of an intelligent agent, including the current task allocation state, the state of the agent, and information about surrounding nodes. Define an action space that represents the task allocation behavior that an agent can choose, such as selecting to execute a task or collaborating with other agents. Use RL algorithms (such as deep Q network, actor–critic algorithm, etc.) to learn task allocation strategies, so that agents can choose the best action according to the current state.

Implement Task Allocation and Collaboration: According to the task allocation strategy learned from the RL algorithm, the agent selects the optimal task allocation behavior according to the current state. Intelligent agents can achieve collaborative work of tasks through communication and coordination. Update task status and agent status in real-time, and use them as input for the next time step to continue task allocation and collaboration.

By combining GNN and RL, agents can better understand the complexity of task allocation and the relationship between agents, and learn better task allocation strategies. The specific implementation and details need to be adjusted and optimized according to specific application scenarios and requirements.

GNNs offer significant advantages by effectively capturing the associations and interactions between agents and tasks in dynamic environments. By learning embedded representations of nodes and edges within graph structures, GNNs can model complex relationships and apply this knowledge to decision making in task allocation. A particularly valuable feature of GNNs is their ability to handle evolving graph structures, where node and edge topologies dynamically change over time. This adaptability is critical for real-world applications, ensuring robust performance in varying scenarios.

In this study, GNNs are leveraged to encode the characteristics of agents and tasks, employing advanced architectures like graph convolutional networks (GCNs) and graph attention networks (GATs). These networks extract feature representations from graph structures, as illustrated in Fig. 2. GATs, in particular, incorporate attention mechanisms that refine the relationships between agents and tasks, focusing on critical information to enhance task allocation. This attention-based approach enables the model to prioritize connections dynamically, leading to improved decision making in task assignment.

To augment the GNN framework, an RL methodology is integrated, providing adaptive and dynamic task allocation strategies. In this framework, deep RL algorithms, such as deep Q-networks (DQNs) or policy gradient methods, guide agents to learn optimal allocation strategies through trial-and-error interaction with the environment. The action space comprises potential task allocation decisions, while a custom reward function evaluates these decisions based on task completion efficiency and overall system performance.

The model employs a dual-level attention mechanism to further enhance its representational power. A global attention mechanism captures overarching dependencies within the graph, while a local attention mechanism focuses on immediate, task-specific relationships. Techniques, such as self-attention and multihead attention are utilized to adjust attention weights dynamically, enabling the model to prioritize the most relevant connections within the graph structure.

Additionally, the model integrates a cyclic feature reconstruction process, which refines input representations through iterative attention updates, enhancing the quality of encoded information. This mechanism complements the dynamic state-update process, where agent states are continuously refined based on task allocation strategies and environmental feedback. These state updates serve as inputs for the RL algorithm, promoting continuous learning and improving task allocation strategies over time.

To ensure adaptability, a self-optimization mechanism is incorporated, allowing the model to adjust its parameters in response to changing task scenarios and agent characteristics. This mechanism employs advanced optimization techniques, including evolutionary strategies and genetic programming, to fine-tune model parameters based on feedback from the environment. This ensures robust performance across diverse task allocation scenarios.

The proposed GMATANN employs a heterogeneous graph representation, where nodes are classified into two distinct types—agents and tasks. This representation is essential for capturing the unique roles and attributes of agents and tasks in the allocation process. The GMATANN architecture incorporates specialized embedding layers for agent and task nodes, effectively preserving the unique characteristics of each node type while enabling meaningful interactions between them. Heterogeneity-aware attention mechanisms dynamically calculate attention scores to model relationships between connected nodes accurately.

To maintain consistency, node features are normalized separately for agents and tasks before inputting them into the GNN. Type-specific aggregation functions are used to combine

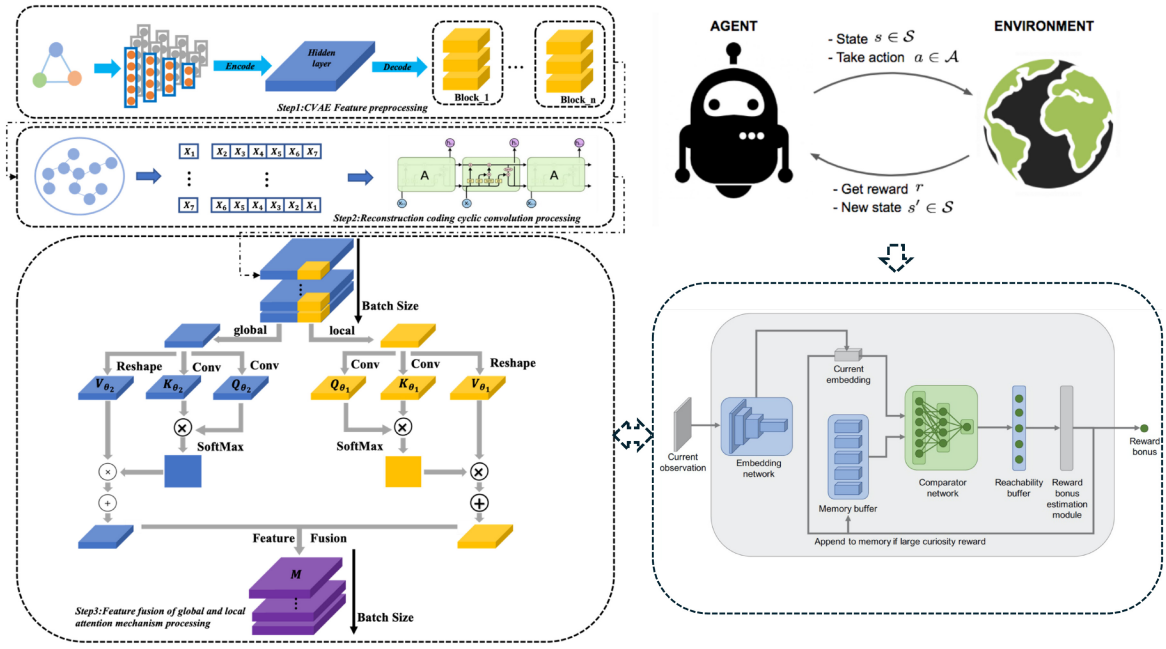


Fig. 2. Architecture of multiagent task allocation system inspired by RL principles.

information from neighboring nodes, preserving the diversity of feature dimensions while enabling seamless interactions. These mechanisms collectively empower the GMATANN model to handle the complexities of heterogeneous graphs, ensuring accurate representation and effective interaction between agents and tasks in the task allocation process.

A. Time-Series Conditional Variational Self-Encoder

The variational self-coder is a crucial component of our proposed model, designed to enhance the signal reconstruction process by automatically learning the coding and decoding of input data. Unlike conventional neural networks that primarily focus on direct feature extraction, our variational self-coder uniquely targets the extraction of hidden features from the data, enabling a more sophisticated representation learning process. The essence of this approach lies in its ability to create a mapping from these hidden features to the generated target data model, thereby improving the accuracy and reliability of the reconstructed signal.

Our variational self-coder differs from existing models in several key ways as follows.

Enhanced Feature Extraction: Unlike traditional autoencoders, which often overlook subtle yet critical data features, our model incorporates a variational approach that extracts potential latent variables more effectively. This allows for a richer and more nuanced understanding of the underlying data structure.

Gaussian Noise Constraint: By introducing a Gaussian noise constraint to the encoding process, our model ensures that the extracted features conform to a Gaussian distribution. This constraint not only enhances the robustness of the model against noise but also improves the generalization capability, which is often a limitation in previous methods that do not incorporate such probabilistic elements.

Improved Mapping from Latent Features: The mapping constructed from latent features to the target data is more precise in our approach, as it leverages the variational self-coder's ability to focus on plausible variables extracted from the original data. This precision in mapping sets our method apart from traditional encoders, which may not fully capture the complexity of the data's hidden features.

By comparing our variational self-coder with other models, we demonstrate how these innovations lead to superior performance in signal reconstruction and representation learning, particularly in complex, noisy environments.

The variational autoencoder (VAE) is a variant of the autoencoder that assumes the hidden variables follow a Gaussian distribution. The encoder learns both the mean and variance of this distribution, using them to generate a hidden layer vector representation Z and sample the hidden variables to reconstruct the target data x. VAE captures not only structural and attribute-based nonlinear similarities but also the underlying data distribution, making it well-suited for tasks involving sparse network representation learning. However, VAE is a self-supervised model that does not incorporate additional information about the data. The main structure is shown in Fig. 3, where the network consists of two components: 1) an encoder G with parameter β and 2) a decoder D with parameter δ, as described in

$$\begin{aligned}\text{Loss}(\phi,\theta,x) = E_{q_\beta(z|x)}\big[&\log P_\delta(x|z) \\ &-D_{\text{KL}}\big(q_\beta(z|x)\big)|P_\delta(z)\big].\end{aligned} \tag{1}$$

$E_{q_\beta(z|x)}[\log P_\delta(x|z)]$ represents the expectation of the log-likelihood of the reconstructed data x given the latent variable z, under the approximate posterior distribution $q_\beta(z|x)$. It quantifies how well the reconstructed data matches the input data and is central to the reconstruction process. $D_{\text{KL}}(q_\beta(z|x))|P_\delta(z)$ is the Kullback–Leibler (KL) divergence,

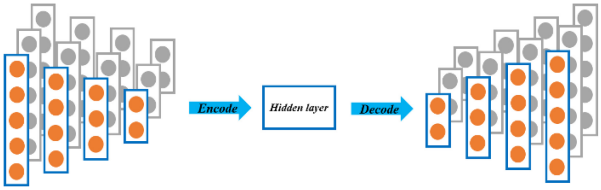


Fig. 3. Composition of the variable division self-encoder.

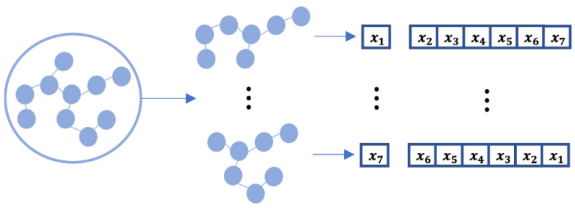


Fig. 4. Variable division self-encoder.

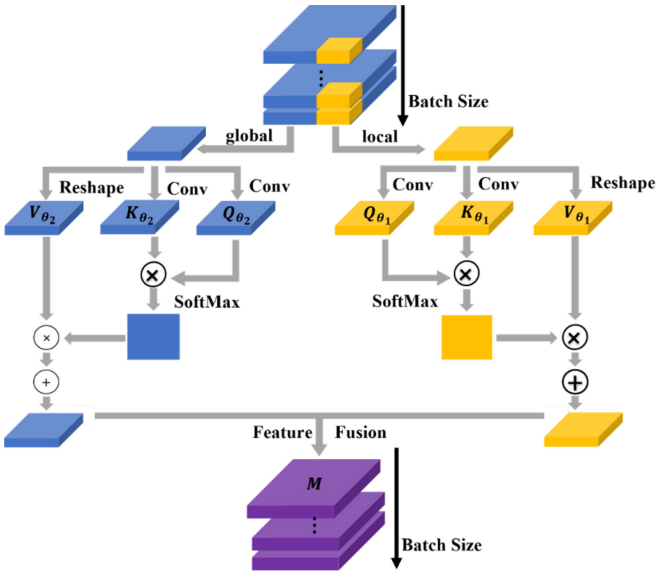


Fig. 5. Global-local fusion feature attention mechanism.

which measures the difference between the approximate posterior distribution and the prior distribution. $q_\beta(z|x)$ denotes the feature encoding from the image input to the hidden space, and $P_\delta(x|z)$ represents the encoding from the hidden space to the data output. The conditional variational self-coding neural network is optimized by adding a label vector constraint to the variational self-coder, allowing the model to process data according to a specified class. The label vector is represented in one-hot encoding format.

There are three various agents, and the labels of these agents need to be encoded uniquely and thermally. The conditional variational self-encoder can add the labels of the data or other prior knowledge as conditions in the encoder and decoder to model the hidden variable z and the data x under conditional probability.

In the design of the conditional variational self-encoder, in order to successfully interface with the subsequent LSTM recurrent neural network, this article optimizes the conditional variational self-encoder and proposes a conditional variational self-encoder with temporal-type processing, so that the data processed by the encoder can be connected to the LSTM recurrent neural network, and the feature images of each image after passing through the self-encoder are reordered temporally, and the data of each feature value in them are rearranged to prepare for subsequent feeding into the recurrent neural network. Fig. 4 shows the structure of the autoencoder designed in this article.

B. Global-Local Fusion Feature Attention Mechanism

The attention mechanism can see the specific task as having three parts to complete, namely Q, where Q denotes the task to be queried, K and V denote the corresponding key-value pairs, and the task is to find the corresponding V value in K using Q.

The model [19] proposed that having representations that are suitable for a specific task and data domain can significantly improve the learning success and robustness of the training model. In this article, we construct attentional mechanisms for high-level representations extracted by VAE to enable the meta-learner to focus on target representations that are more beneficial to the current learning task in the global information.

The self-attention mechanism plays a similar role to the human visual attention mechanism by filtering out some of the critical information from a large amount of information and focusing on this important information. Equation (1) represents the computational formula of the attention mechanism

$$\begin{aligned} D_{\mathcal{V}} &= \text{Attention}(Q_t, k, \mathcal{V}) \\ &= \text{SoftMax}\left(\frac{(Q_t, k_s)}{\sqrt{d_k}}\right)\upsilon_s \\ &= \sum_{s=1}^{m} \frac{1}{z} \exp\left(\frac{(Q_t, k_s)}{\sqrt{d_k}}\right)\mathcal{V}_S \end{aligned} \tag{2}$$

where z is the normalization factor, Q_t represents a matrix containing t query conditions, k_s is a proof containing s key values, d_k is the dimension of each query condition in Q_t, $\mathcal{V}_s$ is a value containing s elements, υ_s is an element in $\mathcal{V}_s$, and $D_{\mathcal{V}}$ is the attention result.

Fig. 5 illustrates the internal structure of the global-local attention model in this article. The module selectively emphasizes certain features by analyzing the total features of the input data and capturing the feature dependencies between the global and local aspects of the picture.

The attention mechanism module is constructed by the hidden features δ generated by the self-encoder, which can adaptively integrate the focused information between local features and global features, so that the model architecture can notice more useful features and perform the classification work excellently. b is the data processed in each batch, h and w are the length and width of the image, c is the number of channels, this article divides the attention mechanism module into global and local feature. In this article, the attention mechanism module is divided into global and local features, and the input data are processed separately, with local features processed using θ_1 and global features processed using θ_2.

Among them, Q and K are integrated across channels by $1*1$ convolution to derive new feature maps, and in both local and global attention mechanism feature selection processing, they are normalized by softmax to obtain attention probability values, so that the design can highlight the role of key feature maps and reduce the impact of redundant features on the overall classification performance

$$\begin{cases} Q_{(\theta_1,\theta_2)} = \mathrm{Conv}(Q(\delta;\theta)) \\ K_{(\theta_1,\theta_2)} = \mathrm{Conv}(Q(\delta;\theta)), \theta_1, \theta_2 \in \theta \\ \mathcal{V} = \mathrm{Reshape}(\delta) \end{cases} \tag{3}$$

$$\partial_{ij} = \frac{\exp\left(Q_i \cdot k_j\right)}{\sum_{t=1}^{C} \exp\left(Q_i \cdot k_j\right)}. \tag{4}$$

The weight coefficients calculated by softmax are weighted and summed with the original features, and then a constraint factor τ is introduced to adjust the feature output to obtain a highly discriminative feature output expression

$$M_j = \tau \sum_{i=1}^{C} \left(\partial_{ij} \cdot \mathcal{V}_i\right) + \delta_j. \tag{5}$$

The global and local feature attention mechanism outputs are derived by the above method, and then the final feature representation is derived by deeply fusing the global and local feature outputs.

In the case of hard attention, the generated hotspot map is in the form of a binary mask, where the important feature regions are 1 and the remainder are 0. There is only one specific region in the attention region, not the whole image.

By incorporating the mask mode into the attention mechanism, the model can effectively handle input sample sequences of varying lengths by preprocessing them accordingly (padding with 0 for short sequences and phase information for long sequences). This integration improves computational efficiency. Additionally, the hard attention mechanism analyses the model's focused region, allowing it to prioritize the core region and achieve additional performance enhancements. One notable advantage of visual attention models, like hard attention, is their ability to accurately analyze regions without the need for explicit bounding boxes to guide the focus of attention.

C. Feature Fusion Process

The GMATANN model employs a sophisticated feature fusion mechanism to combine the information from different types of nodes—agents and tasks—within the heterogeneous graph. The fusion process is crucial for ensuring that the model accurately captures the complex relationships between agents and tasks, which is necessary for effective task allocation. Fig. 6 shows the structure for GMATANN model's feature fusion process.

1) *Node Embedding:* Each agent and task node in the graph is first represented by its respective feature vector. For agent nodes, these features are derived from the agent's variational self-coder, which encodes the agent's state, capabilities, and context into a low-dimensional vector. For task nodes, features are based on the task's requirements, urgency, and other relevant attributes.

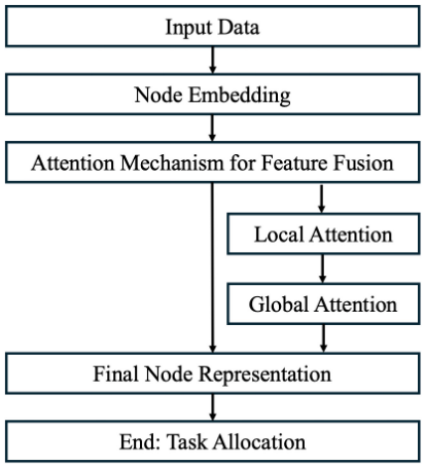


Fig. 6. Flowchart structure for the GMATANN model's feature fusion process.

2) *Attention Mechanism for Feature Fusion:* The GMATANN model utilizes a global-local attention mechanism to perform feature fusion. This mechanism computes attention weights for each node, highlighting the most relevant features depending on the node's context within the graph. The attention mechanism operates in two stages as follows.

1) *Local Attention:* Focuses on the immediate neighborhood of a node, aggregating features from directly connected nodes (e.g., agents directly linked to tasks).
2) *Global Attention:* Captures broader, system-wide dependencies by considering the influence of more distant nodes within the graph. This ensures that the model accounts for both local interactions and global dependencies in the task allocation process.

3) *Weighted Feature Aggregation:* The attention scores are used to weight the features from neighboring nodes, which are then aggregated to update the node's representation. This fusion process allows the GMATANN model to generate a comprehensive feature vector for each node that encapsulates both its intrinsic properties and its relational context within the graph.

D. Reinforcement Learning Integration

RL is integrated into the GMATANN model to optimize the task allocation strategy based on the dynamic environment and the evolving state of the multiagent system.

1) *State Representation:* The state in the RL framework is represented by the current configuration of the graph, including the feature vectors of all nodes after the feature fusion process. This state encapsulates the current task allocations, the status of each agent, and the relationships between agents and tasks.

2) *Action Space:* The action space consists of possible task allocations that an agent can undertake. Each action corresponds to assigning a specific task to an agent or a group of agents. The GMATANN model explores this action space to determine the most effective allocation strategy.

3) *Reward Function:* The reward function is designed to guide the learning process by providing feedback on the effectiveness of the task allocation. It considers factors, such

as task completion time, resource utilization, and overall system efficiency. The reward function encourages actions that lead to efficient and timely task completion while penalizing inefficient or suboptimal allocations.

4) *Learning Process:* The GMATANN model uses deep RL techniques, such as DQN or actor–critic methods, to learn the optimal task allocation policy. The model iteratively updates its policy by interacting with the environment, selecting actions based on the current state, and receiving rewards. Over time, the GMATANN model learns to allocate tasks in a way that maximizes the cumulative reward, reflecting an optimal allocation strategy.

5) *Policy Optimization:* The RL process continuously refines the task allocation policy. The GMATANN model adjusts its action-selection strategy based on the observed outcomes and rewards, ultimately converging on a policy that effectively balances the tradeoffs involved in multiagent task allocation.

III. Problem Descriptions and Assumption Scenarios

Heterogeneous multiagent task allocation is a fundamental challenge in dynamic environments where diverse agents, such as AAVs, UGVs, and USVs, must collaborate to achieve various tasks. This study focuses on scenarios inspired by real-world applications, such as urban disaster response and logistics, where tasks involve surveillance, rescue, and transportation across different domains, including streets, buildings, and water bodies. The problem is modeled as a graph-based task-agent relationship, where agents and tasks are represented as nodes, and edges encode compatibility, dependencies, and collaboration requirements. For example, street areas may require patrol and cargo transportation tasks handled by ground-based agents, while building areas often involve monitoring or rescue tasks requiring aerial capabilities, and water areas demand agents capable of navigation on water surfaces. These task-agent relationships are modeled with attributes, such as task urgency, agent capability, and spatial constraints, providing a structured and flexible representation that directly informs the experiments.

To simplify the analysis, we assume that agents possess heterogeneous capabilities tailored to specific tasks, such as AAVs for aerial tasks and UGVs for ground tasks, and that task priorities and resource constraints vary dynamically, necessitating adaptive allocation strategies. Furthermore, we assume that agents can communicate and coordinate effectively through shared graph-based state representations. These assumptions align with the experimental setups, where the proposed model is evaluated for its ability to dynamically adapt task assignments, optimize performance across diverse scenarios, and handle complex task-agent dependencies. By abstracting the task allocation problem into a graph structure and emphasizing practical applications, this section establishes a clear connection between the problem definition and the experimental validation of the model, streamlining the narrative and improving the readability of this article.

The scenarios designed in this article are abstracted from real-world industrial applications, specifically focusing on urban disaster response and logistics, where heterogeneous multiagent systems, such as AAVs, UGVs, and USVs are increasingly deployed [27], [28], [29]. These scenarios are not hypothetical but are derived from actual challenges faced in industries like emergency management, urban planning, and logistics operations [30], [31]. For example, the task environment and requirements modeled in this article reflect the operational needs observed in disaster-stricken urban areas, where rapid and coordinated responses are critical for saving lives and minimizing damage.

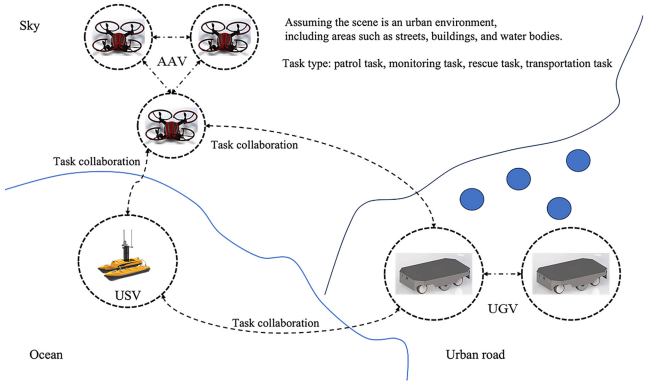


Fig. 7. Assumption scenarios for heterogeneous multiagent tasks.

To ensure the relevance and applicability of our research, the task assumption scenarios include detailed task environment modeling, target task modeling, collaborative modeling of heterogeneous multiagent systems, and constraint condition modeling [32], [33]. These elements are based on the operational demands of real-world scenarios, such as coordinating multiple types of vehicles for rescue and supply missions in a congested urban environment. Fig. 7 illustrates these scenarios, showing how agents with different capabilities are deployed to achieve complex, interdependent tasks that mirror the challenges found in real-life operations.

The specific hypothetical scenario is as follows.

1) *Street Area:* This is the main road network of the city, mainly used for traffic flow. In street areas, patrol and transportation tasks may be involved. The patrol task is to maintain the safety and order of the city, requiring intelligent agents to patrol and monitor the streets. The transportation task is to transport goods from one location to another, and the intelligent agent needs to navigate autonomously on the street to complete the transportation of goods.

2) *Building Area:* This is a group of buildings in a city, including residential, commercial buildings, factories, etc. In the construction area, monitoring and rescue tasks may be involved. Monitoring tasks require intelligent agents to monitor buildings, such as using drones to monitor high-rise buildings. The rescue task is to respond to emergency situations, such as fires or natural disasters, and the intelligent agent needs to perform rescue operations in the building area.

3) *Water Area:* This is the water body of a river, lake, or ocean in a city. In water areas, rescue and transportation tasks may be involved. Rescue tasks require intelligent agents to perform rescue operations in water, such as using unmanned ships for personnel rescue. The transportation task is to

transport goods from one shore to another, and the intelligent agent needs to complete the transportation of goods in the water.

4) Each region may have different characteristics and complexity, requiring intelligent agents to possess corresponding capabilities to complete tasks. In urban area modeling, we also need to consider factors, such as map information, traffic rules, and environmental obstacles, which can affect the navigation and task execution of intelligent agents.

A. Assumption Scenarios of Heterogeneous Multiagent Tasks

Task environment modeling: Scenario description: assuming the scene is an urban environment, including areas, such as streets, buildings, and water bodies. Map modeling: design a city map that includes street networks, building locations, and water areas for the navigation and movement of intelligent agents. Sensor modeling: design appropriate sensors for each intelligent agent, such as cameras for unmanned vehicles, cameras and radar sensors for drones, and laser sensors for unmanned ships, to monitor and collect environmental information [35], [36].

Modeling of task objectives: Patrol tasks: set the target area for patrol tasks, such as specific street areas, and require intelligent agents to patrol and report any abnormal situations. Monitoring task: specify the target area to be monitored, such as important buildings or public places, and require intelligent agents to conduct real-time monitoring and data collection. Rescue task: set the target area for the rescue task, such as a building where a fire occurs, and require the intelligent agent to locate and rescue trapped personnel. Transportation task: set the starting point and destination of the transportation task, and require the intelligent agent to transport goods from one location to another.

Collaborative modeling of heterogeneous multiagent systems: Unmanned vehicles: capable of autonomous navigation and transportation, capable of performing patrol and transportation tasks in street areas. Drones: equipped with aerial monitoring and rapid response capabilities, capable of performing monitoring and rescue tasks in building areas. Unmanned vessel: equipped with water transportation and search capabilities, capable of performing rescue and transportation tasks in water areas. Collaborative model: design communication and collaboration methods between intelligent agents, such as unmanned vehicles and unmanned ships sharing transportation task information, and unmanned vehicles and drones sharing monitoring data.

B. Overall System Architecture of Intelligent Agent

The overall system architecture of intelligent agent formation collaborative task allocation can include the functions and interaction modes of the following modules: command center, unmanned ship, AAV, unmanned vehicle, and task allocation algorithm model. In this system, the command center serves as the center of the entire system, responsible for task planning, allocation, and coordination. Unmanned ships serve as command centers, providing environmental awareness and communication capabilities, while also assuming the role of command centers. Drones and small vehicles act as executing agents, receiving task assignments and instructions through communication with the command center, and executing tasks.

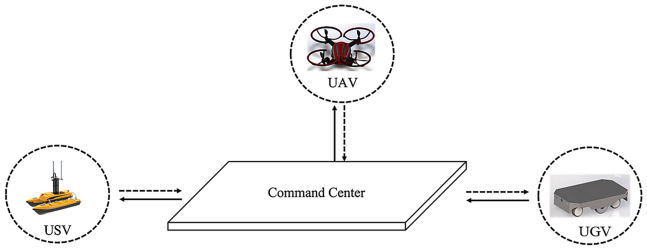


Fig. 8. Interactive method for task allocation in intelligent agent collaboration

The task allocation algorithm model obtains task requirements and agent status information from the command center, calculates the optimal task allocation scheme, and returns the results to the command center to support decision making and task scheduling.

The main functions of the command center are to receive task requirements and environmental information. Design a task allocation algorithm model and calculate the optimal task allocation scheme. Distribute tasks to intelligent agents and send instructions. Receive status information and task execution results from intelligent agents. Perform task scheduling and resource optimization based on feedback information.

C. Interactive Method for Task Allocation

Among them, the main interaction methods include: receiving task requirements and environmental information; inputting task requirements, environmental information, and agent status. *Design Task Allocation Algorithm Model:* Design an optimization algorithm model based on task requirements, environmental information, and the characteristics of agent capabilities. For example, integer linear programming (ILP) or graph theory based algorithms can be used for task allocation and resource optimization. *Assign Tasks and Send Instructions:* Convert the task allocation results into instructions and send them to the corresponding intelligent agents. *Receive Status Information and Task Execution Results:* Through bidirectional communication with the agent, receive the status information and task execution results of the agent. *Task Scheduling and Resource Optimization:* Based on feedback information, dynamically adjust task allocation and resource allocation strategies to improve collaborative efficiency.

The characteristics of USV are the ability to travel on water, suitable for use as a command center, and providing environmental awareness and communication capabilities, as shown in Fig. 8. *Main function:* Provide the construction and maintenance of the command center. Obtain environmental information and transmit it to the command center.

1) Receive tasks assigned by the command center and transmit instructions to the corresponding intelligent agents. Exchange data and transmit commands with other intelligent agents. The main interaction methods include: environmental information transmission: unmanned ships obtain environmental information through sensors and transmit the information

to the command center for task allocation. It can be expressed using mathematical formulas as: $E_{\mathrm{USV}} = \mathrm{Sensors(USV)}$, where E_{USV} represents the environmental information obtained by unmanned ships, and Sensors(USV) represents the sensor data information of unmanned ships. Command transmission: the unmanned ship receives task instructions assigned by the command center and transmits the instructions to the corresponding intelligent agent. It can be expressed as: Instructions(USV) = Command Center $\rightarrow$ USV, where Instructions(USV) represents the instructions received by the unmanned ship, and Command Center $\rightarrow$ USV represents the instructions sent by the command center to the unmanned ship. Data exchange and command transmission: Unmanned ships communicate with other intelligent agents for data exchange and command transmission. It can be expressed using a mathematical formula: Data(USV) = USV $\leftrightarrow$ AAV/AGV, where Data(USV) represents the data exchanged between unmanned ships and other intelligent agents, and AAV/AGV represents AAVs or unmanned vehicles.

2) *AAV Features:* It has flight capability and is suitable for reconnaissance, strike, and air support missions. The interaction mode is command reception: the drone receives tasks and commands assigned by the command center. The formula can be expressed as: Instructions(AAV) = Command Center $\rightarrow$ AAV, where Instructions(AAV) represents the commands received by the AAV, and Command Center $\rightarrow$ AAV represents the commands sent by the command center to the AAV. Task execution: drones perform reconnaissance, strike, and air support tasks based on received instructions. The formula can be expressed as: Mission(AAV) = Execute(AAV, Instructions(AAV)), where Mission(AAV) represents the task performed by the AAV, and Execute(AAV, Instructions(AAV)) represents the function of the AAV to perform the task according to the instructions. Data exchange and command transmission: drones communicate with command centers and other intelligent agents to exchange data and transmit commands. It can be expressed using mathematical formulas as: Data(AAV) = AAV $\leftrightarrow$ CommandCenter/AGV, where Data(AAV) represents the data exchanged between drones and other intelligent agents, and CommandCenter/AGV represents the command center or vehicle.

3) The characteristics of autonomous ground vehicle (AGV) are: it has ground mobility capability and is suitable for ground strike and support missions. Main function: receive tasks and instructions assigned by the command center. Perform ground strike and support missions. Data exchange and command transmission with command centers and other intelligent agents. Among them, interaction mode: command reception: the car receives tasks and commands assigned by the command center. The formula can be expressed as: Instruction(AGV) = Command Center $\rightarrow$ AGV, where Instruction(AGV) represents the instructions received by the unmanned vehicle, and Command Center $\rightarrow$ AGV represents the instructions sent by the command center to the car.

Task Execution: The vehicle executes ground strike and support tasks based on the received instructions. The formula can be expressed as: Mission(AGV) = Execute(AGV, Instruction(AGV)), where Mission(AGV) represents the task performed by the car, and Execute(AGV, Instruction(AGV)) represents the function of the car to perform the task according to the instructions. Data exchange and command transmission: the car exchanges data and transmits commands through communication with the command center and other intelligent agents. The formula can be expressed as: Data(AGV) = AGV $\leftrightarrow$ Command Center/AAV, where Data(AGV) represents the data exchanged between the car and other intelligent agents, and Command Center/AAV represents the command center or drone.

On this basis, this article needs to determine the task requirements and priorities: the command center determines the tasks to be completed and their priority order based on the task requirements and tactical priorities. Agent reporting capability and availability: Each agent reports its own capabilities, status, and availability, including sensor coverage, weapon load, battery level, and other information. Agent proposes task candidate list: each agent proposes an executable task candidate list based on its own capabilities and task requirements. Task allocation game: the command center uses the core solution concept in Game theory to allocate tasks according to task requirements and the report information of agents. The core solution can ensure the maximum efficiency of task allocation and ensure that no intelligent agent is deprived of poor tasks due to task allocation.

Task allocation and communication: the command center assigns tasks to various agents based on the allocation results, and communicates task information to each agent through the communication system. Task execution and collaboration: each agent executes its assigned tasks and takes collaborative actions when needed. Intelligent agents share information through communication systems to ensure real-time task collaboration and target updates.

D. Graph Multiagent Task Allocation Neural Network Mathematics Definitions

The construction of the original graph for agents and tasks is a crucial step in the proposed GMATANN model. This graph, denoted as $G = (V, E)$, consists of a set of nodes V and edges E. The nodes in the graph represent the agents and tasks, while the edges represent the relationships and interactions between them.

1) Mathematical Definitions:

1) *Nodes:* Let $V = \{\upsilon_1, \upsilon_2, \ldots, \upsilon_n\}$ be the set of nodes where each node υ_i represents either an agent a_i or a task t_i. Specifically, $V_A \subset V$ denotes the set of agent nodes, and $V_T \subset V$ denotes the set of task nodes, such that $V = \upsilon_A \cup \upsilon_T$.

2) *Edges:* The edges $E \subseteq V \times V$ represent the relationships between agents and tasks. Each edge $e_{ij} = (\upsilon_i, \upsilon_j)$ is associated with a weight w_{ij} that quantifies the strength or relevance of the relationship between nodes υ_i and υ_j.

2) Types of Relationships and Dependencies: In the context of heterogeneous multiagent systems, the relationships between agents and tasks can vary, and it is important to define and handle these relationships accurately.

1) *Task-Agent Matching:* The primary relationship in the graph is the matching between tasks and agents. For each task $t_i \in V_T$ and agent $a_j \in V_A$, an edge e_{ij} exists if agent a_j is capable of performing task t_i. The weight w_{ij} on this edge represents the degree of matching between the agent's capabilities and the task's requirements. Mathematically, this can be defined as

$$w_{ij} = f(a_j, t_i) \tag{6}$$

where f is a function that evaluates the compatibility between the agent's capabilities and the task's requirements. This function could consider factors, such as the agent's skill level, resource availability, and the complexity of the task.

2) *Collaborative Task Relationships:* In some cases, tasks are interdependent and require collaboration between multiple agents. These dependencies are modeled by edges between task nodes $t_i, t_j \in V_T$. The weight w_{ij} on the edge e_{ij} between two tasks represents the strength of the collaborative relationship, which could be based on factors, such as the need for synchronization, resource sharing, or sequential task execution

$$w_{ij} = g(t_i, t_j) \tag{7}$$

where g quantifies the interdependency between the tasks.

3) *Agent Collaboration:* Agents may also need to collaborate on certain tasks. This is represented by edges between agent nodes $a_i, a_j \in V_A$. The weight w_{ij} on these edges reflects the degree of collaborative potential between agents, based on factors, such as communication efficiency, resource complementarity, and prior collaboration experience

$$w_{ij} = h(a_i, a_j) \tag{8}$$

where h evaluates the potential for effective collaboration between agents.

The weights w_{ij} in the graph represent different types of relationships depending on the context: task-to-agent, task-to-task, or agent-to-agent. To avoid confusion, it is important to clarify how these weights are defined and used. The weight in the context of task-agent matching describes the degree of compatibility between an agent's capabilities and a task's requirements. This relationship focuses on factors, such as the agent's skills, resources, and the complexity of the task. This type of weight applies only to edges that connect task nodes with agent nodes and reflects their ability to work together effectively.

For task-to-task relationships, the weight quantifies the dependency or collaborative need between two tasks. This may involve synchronization requirements, resource sharing, or sequential execution. These weights apply exclusively to edges connecting task nodes and are used to capture the interdependencies that may influence task execution in the system.

Agent-to-agent weights, on the other hand, represent the collaborative potential between two agents. This type of weight reflects factors, such as communication efficiency, complementary resources, and prior experience working together. These edges only connect agent nodes and are critical in scenarios where multiple agents must coordinate to complete tasks.

To distinguish these relationships, the context in which is used should always be made explicit. This can be achieved by clearly identifying whether the edge connects a task to an agent, a task to another task, or an agent to another agent. Additionally, each type of relationship can be visualized and categorized separately in the graph structure, ensuring that the meaning of is consistent and unambiguous across different scenarios. This approach ensures clarity in interpreting the graph and avoids overlap or confusion between the different types of relationships.

3) Handling Multiple Relationships: When dealing with multiple types of relationships in the graph, the GMATANN model employs a multirelational graph approach where different types of relationships are encoded as separate edge types or by using multidimensional edge weights. Specifically, the graph can be represented as a multilayer graph where each layer corresponds to a specific type of relationship (e.g., task-agent matching, task collaboration, and agent collaboration). The attention mechanism within the GMATANN model then dynamically adjusts the importance of each relationship type based on the current context of task allocation.

4) Calculation of Matching Degree and Collaborative Relationships: The degree of matching between an agent and a task is calculated by evaluating the compatibility function $f(a_j, t_i)$, which could be defined as a weighted sum of various factors, such as

$$\begin{aligned} f(a_j, t_i) = {} & \alpha_1 \times \text{SkillMatch}(a_j, t_i) \\ & + \alpha_2 \times \text{ResourceAvailability}(a_j, t_i) \\ & + \alpha_3 \times \text{TaskComplexity}(t_i) \end{aligned} \tag{9}$$

where α_1–α_3 are coefficients that weigh the importance of each factor.

Similarly, the collaborative relationship between tasks or agents is calculated using functions $g(t_i, t_j)$ and $h(a_i, a_j)$, which may incorporate factors like

$$\begin{aligned} g(t_i, t_j) = {} & \beta_1 \times \text{SynchronizationRequirement}(t_i, t_j) \\ & + \beta_2 \times \text{ResourceSharing}(t_i, t_j) \end{aligned} \tag{10}$$

$$\begin{aligned} h(a_i, a_j) = {} & \gamma_1 \times \text{CommunicationEfficiency}(a_i, a_j) \\ & + \gamma_2 \times \text{Complementarity}(a_i, a_j) \end{aligned} \tag{11}$$

where β_1, β_2, γ_1, and γ_2 are weighting factors.

By clearly defining the nodes, edges, and relationships in the task-agent graph, and using these mathematical definitions and functions, the GMATANN model can effectively capture the complexities of heterogeneous multiagent systems. This structured approach allows for accurate task allocation by considering both individual agent-task matching and intertask and interagent dependencies.

E. Establishment of Heterogeneous Multiagent Model

1) Task Environment Modeling: When modeling the task environment, we need to consider the different characteristics and locations of urban areas, as well as the tasks that may be involved in each area. In this heterogeneous multiagent task

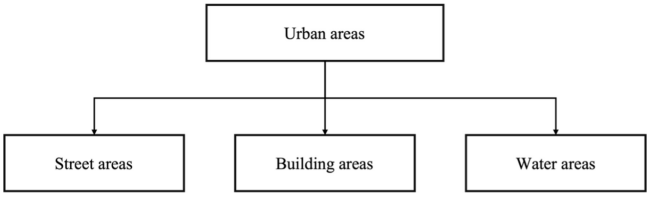


Fig. 9. Environment distribution environment map.

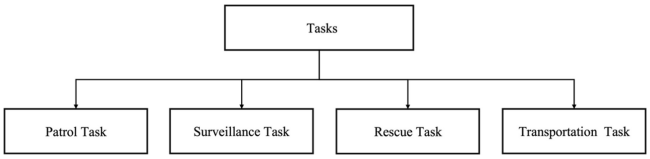


Fig. 10. Task distribution environment map.

scenario, it is assumed that urban areas are divided into the following three types of locations, as shown in Fig. 9.

1) *Street Area:* This is the main road network of the city, mainly used for traffic flow. In street areas, patrol and transportation tasks may be involved. The patrol task is to maintain the safety and order of the city, requiring intelligent agents to patrol and monitor the streets. The transportation task is to transport goods from one location to another, and the intelligent agent needs to navigate autonomously on the street to complete the transportation of goods.

2) *Building Area:* This is a group of buildings in a city, including residential, commercial buildings, factories, etc. In the construction area, monitoring and rescue tasks may be involved. Monitoring tasks require intelligent agents to monitor buildings, such as using drones to monitor high-rise buildings. The rescue task is to respond to emergency situations, such as fires or natural disasters, and the intelligent agent needs to perform rescue operations in the building area.

3) *Water Area:* This is the water body of a river, lake, or ocean in a city. In water areas, rescue and transportation tasks may be involved. Rescue tasks require intelligent agents to perform rescue operations in water, such as using unmanned ships for personnel rescue. The transportation task is to transport goods from one shore to another, and the intelligent agent needs to complete the transportation of goods in the water.

Each region may have different characteristics and complexity, requiring intelligent agents to possess corresponding capabilities to complete tasks. In urban area modeling, we also need to consider factors, such as map information, traffic rules, and environmental obstacles, which can affect the navigation and task execution of intelligent agents.

In summary, task environment modeling involves the division and feature description of urban areas, the types of tasks that may be involved in each area, as well as map information and environmental factors related to the tasks. This modeling can provide a foundation for subsequent intelligent agent task allocation and collaborative decision making.

2) Modeling of Task Objectives: In this section, we will model the specific task objectives in detail, as shown in Fig. 10. Assuming we have four tasks to complete.

Task 1: Patrol Task

Objective: The main objective of the patrol mission is to maintain the safety and order of the city. Intelligent agents need to patrol designated street areas to monitor potential security issues, such as criminal activities and traffic violations.

Requirement: The intelligent agent needs to move according to the predetermined patrol route, observe the surrounding environment, and promptly report and respond to any abnormal situations.

Task 2: Surveillance Task

Objective: The main objective of monitoring tasks is to conduct real-time monitoring and data collection of specific areas. Intelligent agents need to obtain accurate visual information to monitor buildings, public places, or specific targets.

Requirement: The intelligent agent needs to use appropriate sensors, such as cameras on drones, to comprehensively monitor the target area. It also requires real-time analysis and processing of monitoring data in order to detect and report any abnormal activity.

Task 3: Rescue Task

Objective: The main objective of rescue missions is to respond to emergency situations and provide rapid rescue operations. Intelligent agents need to locate and assist trapped individuals, such as victims in disaster situations, such as fires, earthquakes, floods, etc.

Requirement: The intelligent agent needs to search and locate the location of trapped personnel in the building area or water area. It also requires effective rescue operations, such as providing emergency medical assistance and transporting victims to safe areas.

Task 4: Transportation Task

Objective: The main goal of transportation tasks is to transport items from one location to another. Intelligent agents need to complete the collection, transportation, and delivery of goods.

Requirement: The intelligent agent needs to carry out safe and efficient cargo transportation in designated street or water areas. It needs to choose the best path and transportation method according to the task requirements to ensure timely delivery of goods.

F. Collaborative Model Modeling of Heterogeneous Multiagent Systems

In the third section, we will model in detail the collaborative models of three heterogeneous multiagent systems (unmanned vehicles, drones, and unmanned ships). These intelligent agents will collaborate and complete tasks based on task objectives and environmental conditions. The following is a description of the collaborative model modeling for each type of intelligent agent.

1) *Autonomous Vehicle (Function):* Unmanned vehicles have autonomous navigation and transportation capabilities, and perform patrol and transportation tasks in street areas of the city.

Behavior: Unmanned vehicles use map navigation and sensor information to select the best route and driving speed to complete patrol and transportation tasks. It will comply

with traffic rules, avoid collisions, and adjust its driving path in a timely manner to cope with traffic congestion and other situations.

Collaboration: Collaboration with other intelligent agents is mainly reflected in traffic flow regulation and task allocation. Unmanned vehicles can communicate with drones and unmanned ships, share traffic conditions, and task requirements, thereby avoiding conflicts and collisions, and ensuring traffic efficiency.

2) *AAV (Function):* The drone has aerial monitoring and rapid response capabilities to perform monitoring and rescue tasks in the building area.

Behavior: Drones use altitude and speed advantages to quickly reach designated areas for monitoring and searching. It is equipped with high-resolution cameras and sensors, which can collect and transmit monitoring data in real-time. In rescue missions, drones can carry medical supplies or locate trapped individuals.

Collaboration: Drones need to collaborate with unmanned vehicles and unmanned ships to share monitoring data and task information. For example, in rescue missions, unmanned vehicles can update the optimal route based on drone data, while unmanned ships can cooperate to provide support from water areas.

3) *USV (Function):* Unmanned ships have the ability to transport and search on water, and perform rescue and transportation tasks in water areas.

Behavior: Unmanned ships are capable of autonomous navigation in water areas, conducting search and rescue operations. It can carry goods for transportation and perform corresponding tasks in the water, such as rescuing trapped personnel.

Collaboration: The collaboration between unmanned ships, unmanned vehicles, and AAVs includes resource sharing and task allocation. In transportation tasks, unmanned vehicles and unmanned ships can share cargo information according to task requirements and coordinate the best transportation methods. In rescue missions, unmanned ships can provide support and resource scheduling in the water, and cooperate with unmanned vehicles and drones to complete rescue operations.

G. Modeling of Constraints

In the fourth section, we will model the constraints in the heterogeneous multiagent assignment problem in detail. These constraints can ensure the effectiveness and rationality of task allocation and collaborative decision making. The following is a modeling description of the constraint conditions.

1) *Task Requirement Constraint:* Each task has specific requirements, such as patrol tasks requiring a certain number of intelligent agents for patrol coverage, and rescue tasks requiring sufficient resources to execute rescue operations. These task requirement constraints need to be met to ensure the smooth completion of the task.

2) *Resource Constraints:* Each intelligent agent has certain resources and capabilities, such as the transportation capacity of unmanned vehicles, the range and payload capacity of AAVs, and the water mobility ability of unmanned ships. Task allocation needs to consider the resource constraints of agents to ensure that the tasks assigned to them are completed within their capabilities.

3) *Time Constraints:* Task execution usually requires completion within a certain time limit, such as emergency rescue tasks that require response and execution in the shortest possible time. The collaborative decision making and task allocation of intelligent agents need to consider time constraints to minimize task execution time and improve efficiency.

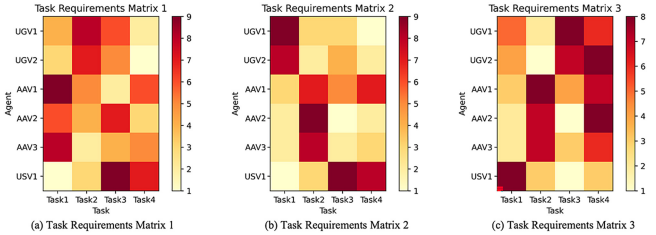
4) *Collision and Conflict Constraints:* Intelligent agents need to avoid collisions and conflicts during task execution to ensure security and collaboration. This can be achieved through technologies, such as path planning, traffic flow regulation, obstacle avoidance algorithms, etc., to ensure safe distance and coordinated action between intelligent agents.

5) *Communication Constraints:* Communication and information sharing are required between intelligent agents to achieve task allocation and collaborative decision making. Communication constraints include communication range, Bandwidth throttling, communication delay and other factors, which need to be considered in task allocation and collaboration models.

IV. Experiments Results

The experimental setup is designed to evaluate the performance of the proposed GMATANN model in various task allocation scenarios inspired by real-world applications, such as urban disaster response and logistics operations. In these experiments, tasks are generated based on realistic profiles with attributes, including type, priority, resource demand, and spatial constraints. For instance, surveillance tasks require high mobility and sensor capabilities, often performed by AAVs, while transportation tasks involve delivering supplies across diverse terrains, suited for UGVs or USVs. Rescue tasks, characterized by high urgency and adaptability, may require collaborative efforts from aerial and ground agents. These task requirements are dynamically adjusted during the experiments to mimic real-time changes, such as evolving priorities or the addition of new tasks, providing a robust test of the model's adaptability.

The experimental environment consists of a heterogeneous multiagent system, where agents, such as AAVs, UGVs, and USVs are assigned specific capabilities based on their types, such as mobility range, payload capacity, and task-specific efficiency. This environment is represented as a graph, where nodes represent tasks and agents, and edges encode compatibility, dependencies, and collaboration relationships. The experiments are conducted in configurations of varying scales, from small setups with ten tasks and five agents to large-scale scenarios involving 100 tasks and 50 agents, to test the scalability and adaptability of the model under increasing complexity. The environment is simulated on a Python-based platform with custom modules for constructing dynamic task-agent graphs, updating task requirements, and evaluating performance.



(a) Task Requirements Matrix 1 (b) Task Requirements Matrix 2 (c) Task Requirements Matrix 3

Fig. 11. Task requirement matrix for multitask and multiagent. (a) Task Requirements Matrix 1. (b) Task Requirements Matrix 2. (c) Task Requirements Matrix 3.

TABLE I
TASK SCENARIO DESIGN TABLE

Experimental group	Task quantity	Task type	Task difficulty level
Group1	5	Patrol, monitoring	simple
Group2	10	Rescue, transportation	medium
Group3	15	Patrol, transportation, rescue	more difficult

Each experiment runs for 500 iterations, during which metrics, such as task completion rate, average resource utilization, and computational efficiency are recorded to assess the model's performance. To ensure a comprehensive evaluation, the GMATANN model is compared with traditional task allocation methods, including centralized optimization techniques and distributed RL algorithms. These benchmarks highlight the advantages of the GMATANN framework in addressing the challenges of dynamic, heterogeneous task allocation scenarios. This enhanced experimental setup description ensures clarity and demonstrates the relevance of the proposed model to both practical applications and theoretical evaluations.

Through the model based on graph attention mechanism neural network proposed in this article, the reliability of multiagent collaborative task allocation in a multitasking environment was tested. Through the combination of multiple AAVs, unmanned vehicles, and unmanned ships, under four different task allocation models, the attention mechanism in our model prioritizes the focus of tasks to the required agents according to different types of tasks, the darker the color, the higher the priority of the task. Fig. 11 shows the task requirement matrix for multitask and multiagent.

In order to evaluate the performance of the algorithm more comprehensively, we will add more task allocation instances and compare them with other optimization algorithms (such as GA and RL). In addition, we will set up experimental groups with different difficulty levels for different task scenarios to gradually increase the difficulty.

Experimental Setup (Design Tables for Multiple Task Scenarios): Create a table to outline the settings for each task scenario. The table should include details, such as the number of tasks, task types, and difficulty levels, as illustrated in Table I.

Describe the Setting of Task Scenarios: Based on the settings in the table, provide a detailed description of the characteristics of each task scenario. For example:

Scenario 1: Contains five patrol and monitoring tasks, distributed throughout the entire urban area, with a lower level of difficulty, as shown in Fig. 12.

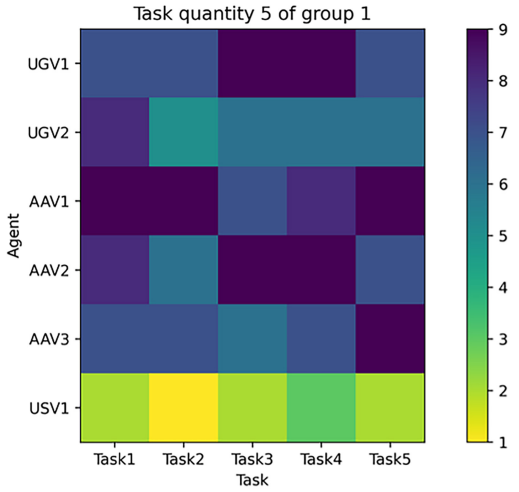


Fig. 12. Task quantify five of group 1.

Fig. 13. Task quantify ten of group 2.

Fig. 14. Task quantify 15 of group 3.

Through the task testing of the first group mentioned above, we can see that the algorithm model has assigned corresponding tasks to different agents, with an accuracy rate of over 90%. By completing simple and difficult tasks, this algorithm model can effectively and quickly allocate tasks.

Scenario 2: Contains ten rescue, transportation, monitoring tasks, with tasks focused on specific buildings and public places, with moderate difficulty levels, as shown in Fig. 13.

From the reality of the above Confusion matrix, we can see that when the difficulty of the task group is moderate, our model still has strong robustness, and three agents have assigned their own tasks, but the accuracy is lower.

Scenario 3: Contains 15 rescue tasks that involve buildings where fires occur, with a high level of difficulty, as shown in Fig. 14.

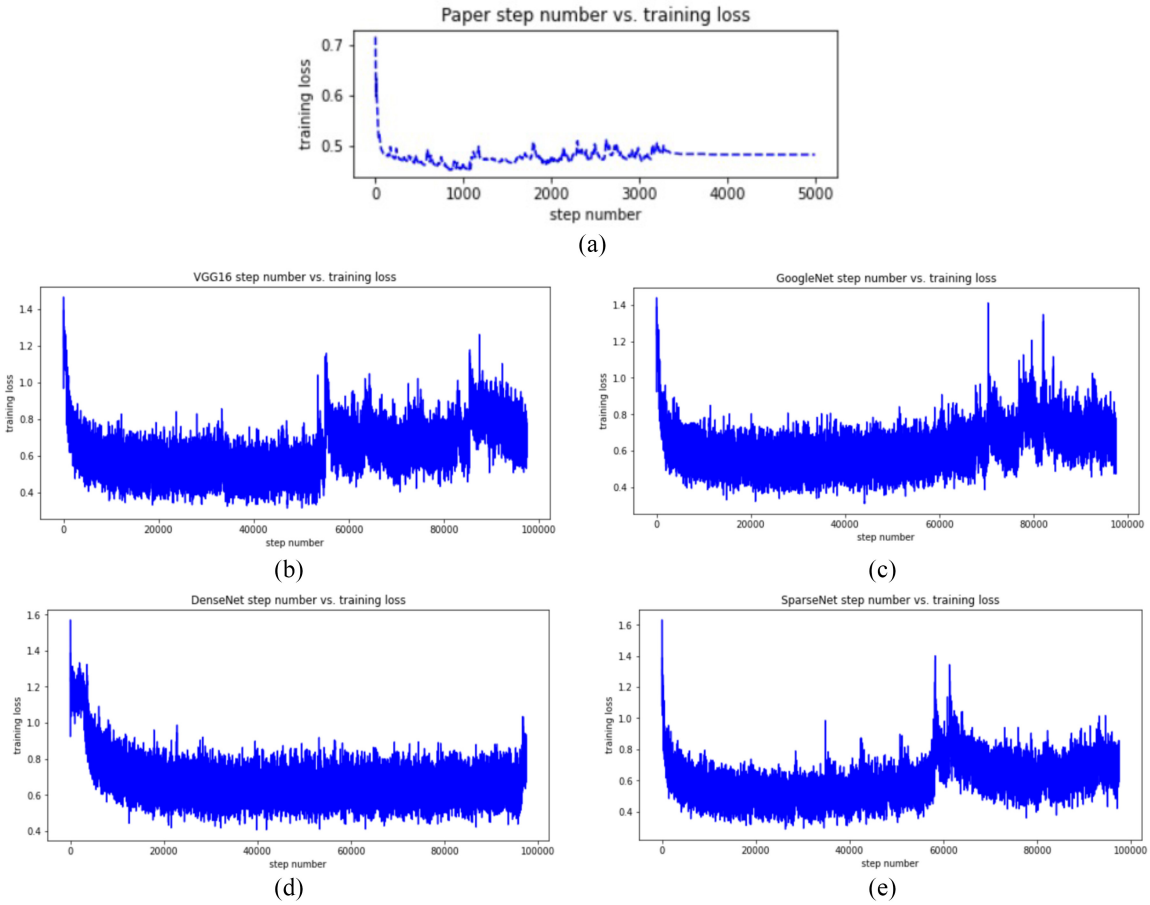


Fig. 15. Comparison of loss functions of different models. (a) GLANNet. (b) VGG16. (c) GoogleNet. (d) DenseNet. (e) SparseNet.

From the reality of the above confusion matrix, we can see that when the task group is the most difficult, our model still has a high robustness, and three agents have assigned their own tasks, but the accuracy is relatively low, but when task four and five are assigned, it still has a high accuracy.

Optimization Algorithm Comparison Experiment: Select other optimization algorithms: select GA, RL, and other optimization algorithms as the benchmark algorithm for comparative experiments.

Comparative Experimental Design: For each task scenario, different optimization algorithms are used for task allocation, and the performance indicators of each algorithm are recorded. For example: For Scenario 1, we will use the designed algorithm, GA, and the RL algorithm to allocate tasks and record performance indicators, such as task completion time and resource utilization.

Similarly, for Scenarios 2 and 3, corresponding comparative experiments were conducted and the performance indicators of the algorithm were recorded.

In the context of multiagent task allocation, it is crucial to use evaluation metrics that align with the objectives of optimizing resource utilization, minimizing task completion time, and improving overall system performance. While metrics, such as AUC, ACC, F1, and recall are commonly used in classification tasks, they do not adequately reflect the effectiveness of task allocation strategies. Therefore, in this study, we focus on metrics that are directly relevant to the multiagent task allocation problem. Average task completion time is selected as a core metric to evaluate the efficiency of the allocation strategy, reflecting how quickly tasks are completed across all agents and capturing the system's ability to minimize delays in task execution, especially in time-critical scenarios like disaster response. Average resource utilization is included to measure how effectively agents use their available resources, such as energy, computational capacity, or operational range, providing insight into the balance between agent workload and resource efficiency. To complement these, average system efficiency serves as a composite metric, integrating aspects of task completion rates, resource utilization, and workload balance to provide a holistic assessment of the system's performance.

These metrics are well-established in task allocation research and are directly tied to the goals of dynamic, heterogeneous multiagent collaboration. To provide a comprehensive evaluation, they are applied across multiple experimental scenarios, including variations in task complexity, the number of tasks and agents, and the dynamic nature of task requirements. The results are presented in detail in Table II, which includes comparative data on average task completion time, average resource utilization, and average system efficiency for each experimental configuration. To further enhance the clarity and accessibility of the results, we have updated Fig. 15 to

include visualizations of these metrics across different scales and scenarios. These visualizations highlight the performance of the proposed GMATANN model compared to traditional methods, showcasing its adaptability and efficiency in diverse conditions. This approach ensures that the evaluation framework is both rigorous and relevant, addressing the critical aspects of multiagent task allocation while improving the presentation and interpretation of the experimental findings.

AUC is the area under the ROC curve, between 0.1 and 1. AUC as a value can be used to visually evaluate the goodness of the classifier, the larger the value the better

$$\text{AUC} = \frac{\sum_{i \in L} \text{Rank}_i - \frac{T(1+T)}{2}}{T \times N} \tag{12}$$

where Rank is the rank, T is the positive class sample, and N is the negative class sample. AUC tends to train a model with as few false positives as possible, i.e., it tends to be conservative in estimation when extrapolating knowledge, while F1 tends to train a model that leaves nothing to chance, i.e., it tends to be aggressive when extrapolating knowledge.

Accuracy is the most common and intuitive evaluation metric for deep learning recognition tasks, which represents the proportion of correctly predicted samples to all samples, as shown in

$$\text{Accuracy} = \frac{\text{TP} + TN}{\text{TP} + TN + \text{FP} + FN} \tag{13}$$

where TP indicates the number of positive samples predicted as positive samples, the sample true value is 0, and the predicted value is also 0; FN indicates the number of positive samples predicted as negative samples, the sample true value is 0, and the predicted value is 1; FP indicates the number of negative samples predicted as positive samples, the sample true value is 1, and the predicted value is 0; and TN indicates the number of negative samples predicted as negative samples, the sample true value is 1, and the predicted value is also 1. The true value of the sample is 1, and the predicted value is also 1.

The precision rate indicates how many of the samples predicted to be positive are true samples, and the formula is as follows:

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}} \tag{14}$$

where TP denotes the number of positive class samples predicted as positive class samples and FP denotes the number of negative class samples predicted as positive class samples.

Recall indicates how many of the positive cases in the sample were predicted correctly, and the formula is as follows:

$$\text{Recall} = \frac{\text{TP}}{\text{TP} + FN} \tag{15}$$

where TP denotes the number of positive class samples predicted as positive class samples and FN denotes the number of positive class samples predicted as negative class samples.

The F1 score is a weighted summed average of the precision and recall rates, with the following equation:

$$F1 = \frac{2\text{Precision} * \text{Recall}}{\text{Precision} + \text{Recall}}. \tag{16}$$

TABLE II
COMPARISON OF EVALUATION INDEXES OF DIFFERENT MODELS

	AUC	ACC	F1	Recall
VGG16	0.896	0.907	0.890	0.885
GoogleNet	0.912	0.915	0.904	0.894
DenseNet	0.921	0.923	0.916	0.905
SparseNet	0.927	0.919	0.898	0.917
Multi-Agent Deep Reinforcement Learning (MADRL)	0.929	0.917	0.915	0.905
Graph-Based Reinforcement Learning (GBRL)	0.930	0.932	0.919	0.910
Hierarchical Attention Networks (HAN)	0.932	0.929	0.914	0.918
Proposed Paper	**0.931**	**0.937**	**0.920**	**0.919**

TABLE III
COMPARISON OF EVALUATION INDEXES OF THIS ARTICLE'S ALGORITHM UNDER DIFFERENT BATCHSIZE OF EVALUATION INDEXES OF DIFFERENT MODELS

	AUC	ACC	F1	Recall
10	0.871	0.891	0.865	0.881
20	0.875	0.893	0.887	0.879
30	0.893	0.906	0.898	0.904
40	0.917	0.911	0.905	0.910
50	0.921	0.907	0.910	0.908
60	0.923	0.914	0.913	0.901
70	0.919	0.915	0.919	0.911
80	0.905	0.903	0.908	0.910
90	0.901	0.896	0.910	0.901

When the F1 score is high, it can indicate that the experimental method is more effective.

The comparative loss function of the dataset used in this article under different models is shown in Fig. 15. It can be seen that the loss function of the algorithm framework proposed in this article can be reduced quickly and can reach a stable state after about 3500 times, while comparing with VGG16, GoogleNet, DenseNet, and SparseNet neural network framework, we can see that the loss function of these models cannot be reduced quickly and cannot be stabilized at a specific value, which will produce a phenomenon similar to numerical explosion, and it can be seen that the GMATmodel proposed in this article has a better performance.

In this article, the evaluation index data of different models were compared and analyzed, as shown in Table II, it can be seen that the GMATmodel architecture proposed in this article can improve the accuracy and recall rate of the model, and has better performance than other more traditional models, with an accuracy rate of 93.1%. model with different batch sizes, and tested the evaluation index values under different groups of small sample batches.

As shown in Table III, the batches were tested from different values of 10, 20, 30, 40, 50, 60, 70, 80, and 90 for the models, and it can be seen that the models all have good performance.

The results present the simulated experiments evaluating the performance of a multiagent system, as shown in Fig. 16, across three different scenarios: 1) urban patrol and monitoring; 2) emergency response and rescue; and 3) maritime logistics and rescue. The results are displayed in three bar charts that depict the average task completion time, average resource utilization, and average system efficiency for each scenario. In the average task completion time chart, there is

a clear increase in task completion time as the complexity of the scenarios increases. The "urban patrol and monitoring" scenario, which is relatively simpler, shows the shortest completion time, averaging around 6 min. The "emergency response and rescue" scenario takes longer, averaging approximately 8 min, reflecting the added complexity of emergency situations that involve more dynamic and urgent tasks. The most complex scenario, "maritime logistics and rescue" has the highest task completion time, averaging about 12 min. This trend highlights the challenges posed by more complex environments and tasks, which naturally require more time to complete. The average resource utilization chart reveals a similar pattern where resource utilization decreases as the scenario complexity increases. The urban patrol and monitoring scenario, which requires less coordination and fewer resources, shows the highest utilization rate, close to 80%. As the complexity increases in the emergency response and rescue scenario, the resource utilization drops to around 73%. The maritime logistics and rescue scenario, which involves complex logistics and coordination across different environments, exhibits the lowest resource utilization at about 65%. This indicates that as tasks become more complex, the system's ability to efficiently utilize its resources diminishes, likely due to the increased demands and constraints imposed by the environment and the nature of the tasks. Finally, the average system efficiency chart reflects a decline in efficiency as the scenario complexity grows. The urban patrol and monitoring scenario maintains the highest efficiency at around 87%, while the emergency response and rescue scenario has a moderate efficiency of approximately 80%. The maritime logistics and rescue scenario shows the lowest system efficiency, around 72%, which aligns with the increased task completion times and reduced resource utilization. This suggests that as the system faces more complex tasks, it struggles to maintain high efficiency, possibly due to the more intricate and resource-intensive nature of the operations required. This analysis underscores the challenges that arise as tasks and environments become more complicated, necessitating more sophisticated strategies for task allocation and resource management to maintain system performance.

The convergence of the proposed GMATANN model is driven by its capacity to iteratively refine task-agent relationships through performance-based feedback. The model employs a dynamic feedback mechanism integrated within the graph attention framework, adjusting attention weights based on task completion status, resource utilization, and system efficiency metrics. This iterative refinement continues until performance metrics stabilize, signaling convergence. Specifically, the model assesses improvements in average task completion time and resource utilization at each iteration, with convergence defined as the point where these metrics exhibit no significant improvement over a predefined number of iterations. This approach ensures that the task allocation strategy reaches an optimal or near-optimal solution.

To evaluate the computational efficiency of GMATANN, we compare it with traditional task allocation algorithms, including centralized optimization methods and distributed RL approaches. The results, summarized in Table III, indicate

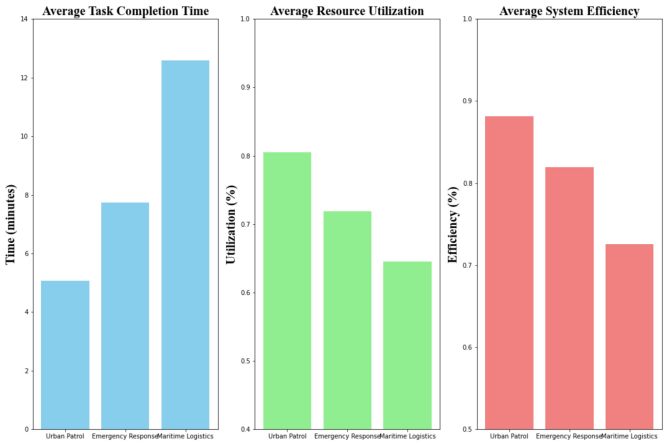


Fig. 16. Analysis of task completion time, resource utilization, and system efficiency in a multiagent system across different complex scenarios.

the computation time required for each algorithm under different experimental configurations: small-scale, medium-scale, and large-scale environments. The GMATANN model exhibits competitive computation times due to its ability to process task-agent relationships in parallel through graph-based operations. In large-scale scenarios, GMATANN achieves significantly lower computation times compared to centralized optimization methods, demonstrating its scalability. Although distributed RL approaches show similar computation times in smaller environments, their performance declines as the complexity of task-agent relationships increases, while GMATANN maintains its efficiency.

V. Conclusion

In this study, we proposed the GMATANN model, addressing key challenges in multiagent task allocation through an innovative integration of graph attention mechanisms and feedback-driven optimization. The results demonstrate the model's ability to efficiently handle dynamic, heterogeneous task allocation scenarios, but several gaps in current research remain that require further exploration. One primary gap is the tradeoff between accuracy and efficiency. Although GMATANN achieves high accuracy in task allocation, it still requires multiple iteration rounds to converge, which may impact real-time applicability in scenarios with stringent time constraints. Another limitation lies in the need for enhanced adaptability to extreme scenarios, such as environments with rapidly evolving task requirements or highly unbalanced agent capabilities. Furthermore, the computational complexity of graph-based models, while competitive compared to traditional methods, poses challenges when scaling to extremely large systems with thousands of agents and tasks.

To address these gaps, future research should focus on developing techniques to accelerate convergence without sacrificing accuracy, such as incorporating advanced optimization strategies or hybrid learning methods. Additionally, integrating real-world constraints, such as communication delays and energy limitations, into the model would enhance its practical applicability. Another promising direction involves leveraging

transfer learning to enable the model to generalize across diverse task allocation scenarios, reducing the need for retraining in new environments. Finally, advancements in distributed computation frameworks can be explored to further improve the scalability of graph-based approaches, enabling efficient deployment in large-scale, real-time applications.

REFERENCES

[1] D. S. Drew, "Multiagent systems for search and rescue applications," *Current Robot. Rep.*, vol. 2, pp. 189–200, Mar. 2021.

[2] J. Wu, J. Zhang, Y. Sun, X. Li, L. Gao, and G. Han, "Multi-UAV collaborative dynamic task allocation method based on ISOM and attention mechanism," *IEEE Trans. Veh. Technol.*, vol. 73, no. 5, pp. 6225–6235, May 2024.

[3] D. Liu, L. Dou, R. Zhang, X. Zhang, and Q. Zong, "Multiagent reinforcement learning-based coordinated dynamic task allocation for heterogenous UAVs," *IEEE Trans. Veh. Technol.*, vol. 72, no. 4, pp. 4372–4383, Apr. 2023.

[4] R. Nishtala, V. Petrucci, P. Carpenter, and M. Sjalander, "Twig: Multiagent task management for colocated latency-critical cloud services," in *Proc. IEEE Int. Symp. High Perform. Comput. Archit.*, 2020, pp. 167–179.

[5] K. Li, S. Wu, Y. Wen, and Y. Wang, "Task allocation of multiagent groups in social networked systems," *IEEE Internet Things J.*, vol. 9, no. 14, pp. 12194–12208, Jul. 2022.

[6] S. Wang, Y. Liu, Y. Qiu, S. Li, and J. Zhou, "An efficient distributed task allocation method for maximizing task allocations of multirobot systems," *IEEE Trans. Autom. Sci. Eng.*, vol. 21, no. 3, pp. 3588–3602, Jul. 2024.

[7] W. Du and S. Ding, "A survey on multiagent deep reinforcement learning: From the perspective of challenges and applications," *Artif. Intell. Rev.*, vol. 54, no. 5, pp. 3215–3238, 2021.

[8] Y. Wu, S. Wu, and X. Hu, "Cooperative path planning of UAVs & UGVs for a persistent surveillance task in urban environments," *IEEE Internet Things J.*, vol. 8, no. 6, pp. 4906–4919, Mar. 2021.

[9] J. Li et al., "Energy-efficient ground traversability mapping based on UAV-UGV collaborative system," *IEEE Trans. Green Commun. Netw.*, vol. 6, no. 1, pp. 69–78, Mar. 2022.

[10] L. P. Qian, H. Zhang, Q. Wang, Y. Wu, and B. Lin, "Joint multidomain resource allocation and trajectory optimization in UAV-assisted maritime IoT networks," *IEEE Internet Things J.*, vol. 10, no. 1, pp. 539–552, Jan. 2023.

[11] F. Ho, A. Goncalves, A. Salta, M. Cavazza, R. Geraldes, and H. Prendinger, "Multiagent path finding for UAV traffic management: Robotics track," in *Proc. 18th Int. Conf. Auton. Agents Multiagent Syst.*, 2019, pp. 131–139.

[12] T. Luo, B. Subagdja, D. Wang, and A.-H. Tan, "Multiagent collaborative exploration through graph-based deep reinforcement learning," in *Proc. IEEE Int. Conf. Agents*, 2019, pp. 2–7.

[13] E. S. Lee, L. Zhou, A. Ribeiro, and V. Kumar, "Graph neural networks for decentralized multiagent perimeter defense," *Front. Control Eng.*, vol. 4, Jan. 2023, Art. no. 1104745.

[14] R. A. Yeh, A. G. Schwing, J. Huang, and K. Murphy, "Diverse generation for multiagent sports games," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2019, pp. 4610–4619.

[15] J. Li, H. Ma, Z. Zhang, J. Li, and M. Tomizuka, "Spatio-temporal graph dual-attention network for multiagent prediction and tracking," *IEEE Trans. Intell. Transp. Syst.*, vol. 23, no. 8, pp. 10556–10569, Aug. 2022.

[16] H. Ma, Y. Sun, J. Li, M. Tomizuka, and C. Choi, "Continual multiagent interaction behavior prediction with conditional generative memory," *IEEE Robot. Autom. Lett.*, vol. 6, no. 4, pp. 8410–8417, Oct. 2021.

[17] W. Qin, Y.-N. Sun, Z.-L. Zhuang, Z.-Y. Lu, and Y.-M. Zhou, "Multiagent reinforcement learning-based dynamic task assignment for vehicles in urban transportation system," *Int. J. Prod. Econ.*, vol. 240, Oct. 2021, Art. no. 108251.

[18] C. Lin, G. Han, T. Zhang, S. B. H. Shah, and Y. Peng, "Smart underwater pollution detection based on graph-based multiagent reinforcement learning toward AUV-based network ITS," *IEEE Trans. Intell. Transp. Syst.*, vol. 24, no. 7, pp. 7494–7505, Jul. 2023.

[19] J. Wiederer, A. Bouazizi, M. Troina, U. Kressel, and V. Belagiannis, "Anomaly detection in multiagent trajectories for automated driving," in *Proc. 5th Conf. Robot Learn.*, 2022, pp. 1223–1233.

[20] T. Westny, J. Oskarsson, B. Olofsson, and E. Frisk, "MTP-GO: Graph-based probabilistic multiagent trajectory prediction with neural ODEs," *IEEE Trans. Intell. Veh.*, vol. 8, no. 9, pp. 4223–4236, Sep. 2023.

[21] J. Peng, W. Chen, Y. Fan, H. He, Z. Wei, and C. Ma, "Ecological driving framework of hybrid electric vehicle based on heterogeneous multiagent deep reinforcement learning," *IEEE Trans. Transp. Electrif.*, vol. 10, no. 1, pp. 392–406, Mar. 2024.

[22] J. Wang, M. Yuan, Y. Li, and Z. Zhao, "Hierarchical attention master–slave for heterogeneous multiagent reinforcement learning," *Neural Netw.*, vol. 162, pp. 359–368, May 2023.

[23] M. Bettini, A. Shankar, and A. Prorok, "System neural diversity: Measuring behavioral heterogeneity in multiagent learning," 2023, *arXiv:2305.02128*.

[24] W. Li, X. Du, J. Xiao, and L. Zhang, "Bipartite hybrid formation tracking control for heterogeneous multiagent systems in multigroup cooperative-competitive networks," *Appl. Math. Comput.*, vol. 456, Nov. 2023, Art. no. 128133.

[25] G. Bao, L. Ma, and X. Yi, "Recent advances on cooperative control of heterogeneous multiagent systems subject to constraints: A survey," *Syst. Sci. Control Eng.*, vol. 10, no. 1, pp. 539–551, 2022.

[26] Z. Gao, L. Yang, and Y. Dai, "Large-scale computation offloading using a multiagent reinforcement learning in heterogeneous multiaccess edge computing," *IEEE Trans. Mobile Comput.*, vol. 22, no. 6, pp. 3425–3443, Jun. 2023.

[27] C. Wakilpoor, P. J. Martin, C. Rebhuhn, and A. Vu, "Heterogeneous multiagent reinforcement learning for unknown environment mapping," 2020, *arXiv:2010.02663*.

[28] O. V. Darintsev, B. S. Yudintsev, A. Y. Alekseev, D. R. Bogdanov, and A. B. Migranov, "Methods of a heterogeneous multiagent robotic system group control," *Procedia Comput. Sci.*, vol. 150, pp. 687–694, Apr. 2019.

[29] A. T. Buyukkocak, D. Aksaray, and Y. Yazıcıoğlu, "Planning of heterogeneous multiagent systems under signal temporal logic specifications with integral predicates," *IEEE Robot. Autom. Lett.*, vol. 6, no. 2, pp. 1375–1382, Apr. 2021.

[30] M. Pallin, J. Rashid, and P. Ögren, "A decentralized asynchronous collaborative genetic algorithm for heterogeneous multiagent search and rescue problems," in *Proc. IEEE Int. Symp. Saf., Security, Rescue Robot.*, 2021, pp. 1–8.

[31] K. Xue et al., "Heterogeneous multiagent zero-shot coordination by coevolution," 2022, *arXiv:2208.04957*.

[32] Y. Rizk, M. Awad, and E. W. Tunstel, "Cooperative heterogeneous multirobot systems: A survey," *ACM Comput. Surv.*, vol. 52, no. 2, pp. 1–31, 2019.

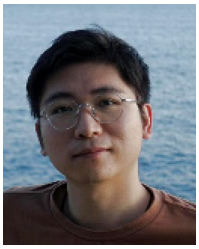
[33] G. Wang, X. Wang, and S. Li, "Signal generator based finite-time formation control for disturbed heterogeneous multiagent systems," *J. Franklin Inst.*, vol. 359, no. 2, pp. 1041–1061, 2022.

Ziyuan Ma (Member, IEEE) received the M.S. degree in navigation, guidance, and control from the College of Automation Engineering, Nanjing University of Aeronautics and Astronautics, Nanjing, China, in 2018, where he is currently pursuing the Ph.D. degree in control science and engineering.

His research focuses on mission planning technology, multiagent systems, and optimization and design of algorithms.

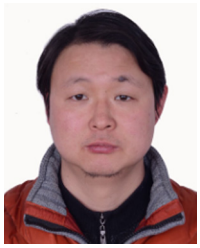
Huajun Gong received the Ph.D. degree from the College of Automation Engineering, Nanjing University of Aeronautics and Astronautics (NUAA), Nanjing, China, in 2000.

He currently serves as a Full Professor with NUAA, where he is the Director of the Personnel Office. In the academic year spanning September 2004 to July 2005, he served as an Academic Visitor with Louisiana State University, Baton Rouge, LA, USA. His research endeavors primarily revolve around multiagent guidance and planning, alongside algorithmic innovations for unmanned systems.



Jun Xiong (Member, IEEE) received the B.S. degree from Nanjing University of Posts and Telecommunications (NUPT), Nanjing, China, in 2014, and the M.S. and Ph.D. degrees from Nanjing University of Aeronautics and Astronautics, Nanjing, in 2017 and 2021, respectively.

He was a Practicum Researcher with the Australian Center for Space Engineering Research, University of New South Wales, Kensington, NSW, Australia. He is currently a Lecturer with the School of Internet of Things, NUPT. His research interests include cooperative navigation and sensing, integrated navigation system, and integrity monitoring.



Xinhua Wang received the Ph.D. degree from the College of Automation Engineering, Nanjing University of Aeronautics and Astronautics, Nanjing, China, in 2012.

He currently serves as an Associate Professor with the School of Automation Engineering. Additionally, he is the Director of the Laboratory of Automation Department, Nanjing University of Aeronautics and Astronautics. He was an Academic Visitor with Louisiana State University, Baton Rouge, LA, USA, from September 2004 to July 2005. His research interests encompass unmanned systems, multiagent mission planning, and flight control.